PATHFINDER X

Inside the Matrix:
Finding Every Shortest Route

Floyd–Warshall Algorithm

Anushka Murkute · PRN 25070521100 · Division B
B.Tech CSE · Symbiosis Institute of Technology, Nagpur
Design & Analysis of Algorithms · Second Year

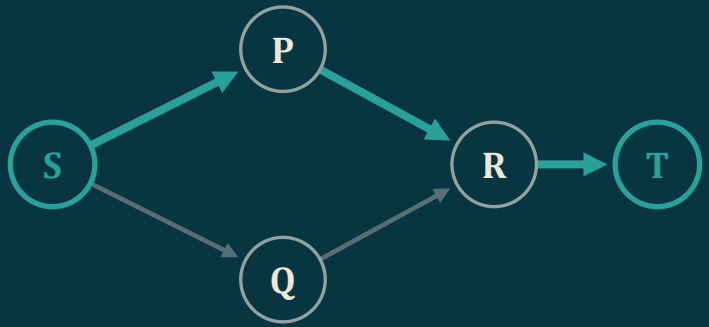
	A	B	C	D
A	0	3	5	6
B	5	0	2	3
C	3	6	0	1
D	2	5	7	0

the matrix we are about to build

Why every pair matters

A map app, a router and a logistics planner all ask the same graph thousands of different questions.

ONE QUESTION



Single-source asks: what is the best S → T? Repeat it from every start vertex to cover all pairs.

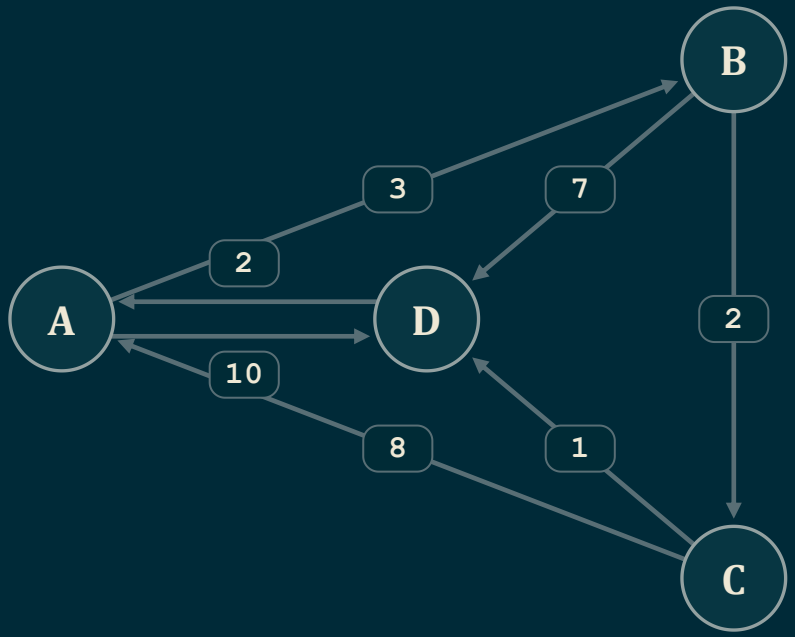
vs

EVERY QUESTION

	A	B	C	D
A	0	3	5	6
B	5	0	2	3
C	3	6	0	1
D	2	5	7	0

All 16 answers, built once. Any “i → j?” afterwards is a single table lookup.

The graph as a matrix



4 vertices · 7 directed, weighted edges

3	**Edge → its weight** Row = from, column = to. A→B costs 3, so cell [A,B] = 3.
0	**Diagonal → 0** A vertex reaches itself at no cost.
∞	**No direct edge → ∞** There is no A→C edge, so [A,C] = ∞ for now.

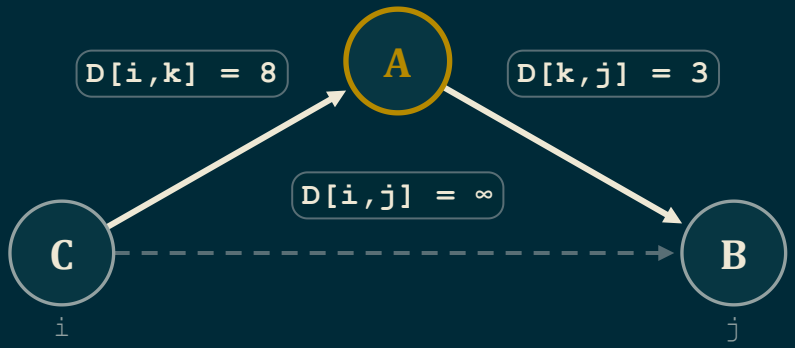
Non-negative weights only (w ≥ 0).
This example is non-negative. Negative edges are legal in general; negative cycles are not.

The recurrence, decoded

$$D^{k}[i, j] = \min(D^{k-1}[i, j] , D^{k-1}[i, \mathbf{k}] + D^{k-1}[\mathbf{k}, j])$$

Can a detour through k beat the best route we already know?

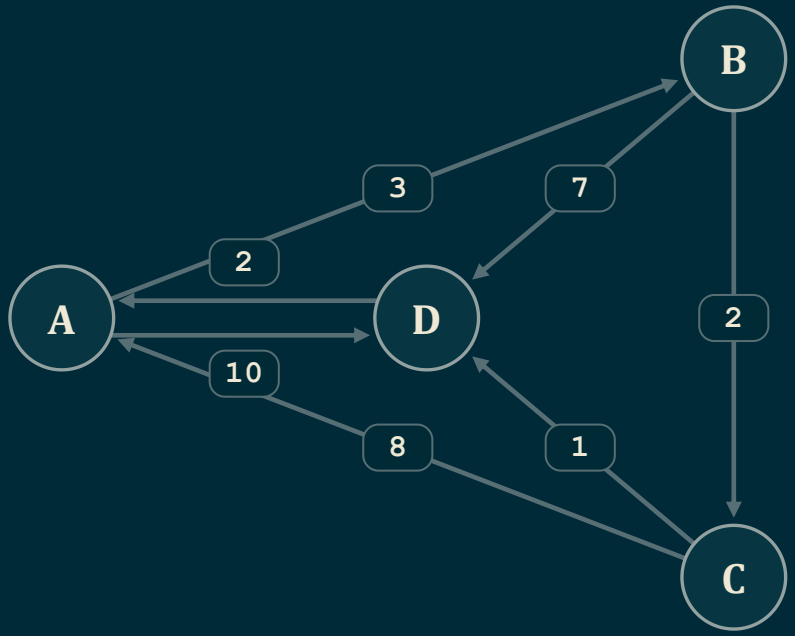
EXAMPLE · i = C, k = A, j = B



`min(∞, 8 + 3) = 11`

`k`	the stop we now allow
`i , j`	start and end vertex
D^{k-1}`[i,j]`	best i→j found so far
`[i,k] + [k,j]`	detour: i→k, then k→j
`min`	keep the shorter one

Initial matrix D^0



D^0 · initial

	A	B	C	D
A	0	3	∞	10
B	∞	0	2	7
C	8	∞	0	1
D	2	∞	∞	0

7 edges → 7 finite · 5 ∞ · 4 zeros

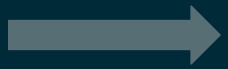
Read every cell as: from row → to column.

Iteration via A

D^0 · before

	A	B	C	D
A	0	3	∞	10
B	∞	0	2	7
C	8	∞	0	1
D	2	∞	∞	0

k = A



Only rows that already reach A can improve: C (8) and D (2).

Checked, not better: C→D stays 1 (8 + 10 = 18).

D^1 · after k = A

	A	B	C	D
A	0	3	∞	10
B	∞	0	2	7
C	8	11	0	1
D	2	5	∞	0

C→B `min(∞, 8 + 3) = 11`

No direct road C→B, but C→A (8) then A→B (3) gives 11.

D→B `min(∞, 2 + 3) = 5`

D→A (2) then A→B (3) gives 5, where none was known.

Iterations via B, then C

D^1 · before B

	A	B	C	D
A	0	3	∞	10
B	∞	0	2	7
C	8	11	0	1
D	2	5	∞	0

B

D^2 · after k = B

	A	B	C	D
A	0	3	5	10
B	∞	0	2	7
C	8	11	0	1
D	2	5	7	0

C

D^3 · after k = C

	A	B	C	D
A	0	3	5	6
B	10	0	2	3
C	8	11	0	1
D	2	5	7	0

HOW TO READ

- amber = the stop k in play
- cyan = shorter route found
- muted = tie or loss, no change

VIA B · $D^1 \rightarrow D^2$

- A→C min(∞, 3+2) = 5
- D→C min(∞, 5+2) = 7
- A→D min(10, 3+7) = 10

VIA C · $D^2 \rightarrow D^3$

- A→D min(10, 5+1) = 6
- B→A min(∞, 2+8) = 10
- B→D min(7, 2+1) = 3

Every shortest distance

D^4 · after k = D (final)

	A	B	C	D
A	0	3	5	6
B	5	0	2	3
C	3	6	0	1
D	2	5	7	0

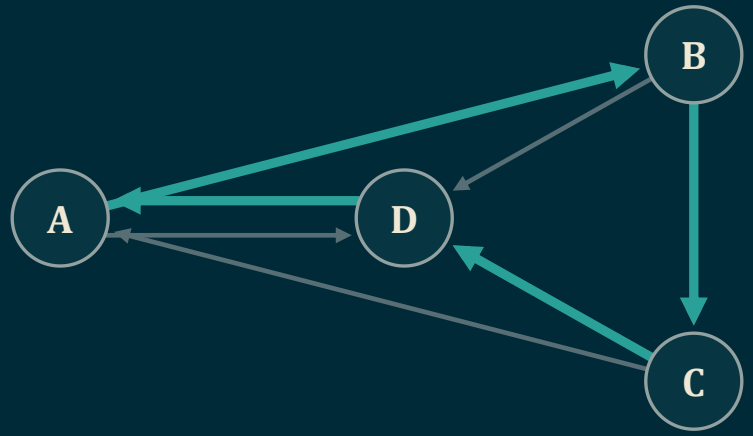
k = D · last stop

B→A min(10, 3+2) = 5

C→A min(8, 1+2) = 3

C→B min(11, 1+5) = 6

ROUTES RECONSTRUCTED FROM THE GRAPH



d(A,D) = 6 A→B→C→D (3+2+1)

d(B,A) = 5 B→C→D→A (2+1+2)

d(C,B) = 6 C→D→A→B (1+2+3)

■ improved in last pass ☐ improved earlier

Pseudocode & complexity

```
for k in 1..V:          # stop
  for i in 1..V:        # from
    for j in 1..V:      # to
      via = D[i][k] + D[k][j]
      if via < D[i][j]:
        D[i][j] = via
        next[i][j] = next[i][k]
```

DISTANCE

D[i][j] answers “how far?”

Filled by the three loops above.

PATH

next[i][j] answers “which way?”

Follow it hop by hop: i → … → j.

$O(V^3)$ — TIME

V × V × V loops · $4^3 = 64$ checks here

$O(V^2)$ — SPACE

one V × V table, updated in place

NEGATIVE-CYCLE CHECK

`D[i][i] < 0`

means i lies on a negative-weight cycle. Here the diagonal stays 0, 0, 0, 0.

Where it matters



Transportation

Travel-cost tables between every pair of stops or depots: built once, queried instantly.



Network routing

Lowest-cost path between every pair of nodes in a small network, ready for routing tables.



Graph analysis

Reachability, graph diameter and closeness scores all read straight from the final matrix.

One graph. Every pair.
All shortest distances.